



SSSMapGeneratorをご購入いただき ありがとうございました!

もしアセットが気に入っていただけたなら、レビューもよろしくお願いします!
願わくばいいものを…

ご意見、ご質問、プログラム改善の提案、追加機能の提案、バグ報告、その他ございましたら下記までご連絡ください

E-mail : sw2.26tks@gmail.com

X : https://x.com/sowa_jp

▼SSSMapGeneratorについて

「SSSMapGenerator」は「Slay the Spire」スタイルのマップを生成できるアセットです
非常に柔軟なカスタマイズが可能で、「Slay the Spire」のようなマップだけでなく、独自のアイデアに基づいたユニークなマップも作成できます

▼SSSMapGeneratorの注意点

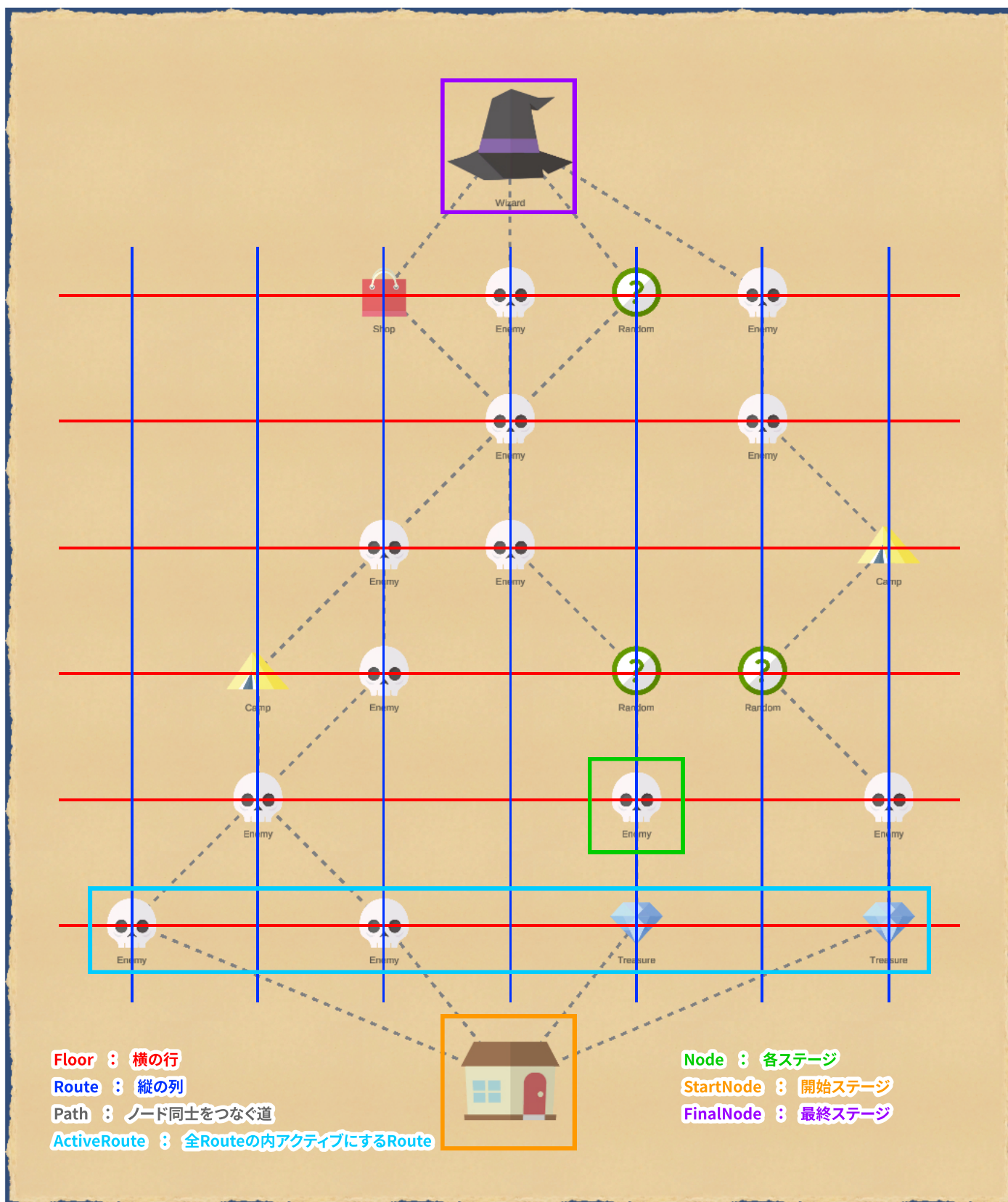
「TextMeshPro」と「InputSystem」を利用しているので事前にインストールしておいてください(必須)

「SSSMapGenerator」には**マップ保存機能はありません**

シングルトンを使用していますのでゲーム中であればシーン移動してもマップは維持されますが、
ゲーム終了後にマップの途中から再開するという事はできませんのでご注意ください
しかし、「seed」を使う事でいつでも同じマップを生成する事はできます

各ノード(敵やお店)の実行には多少プログラムを触る事になりますので、
C#を全く理解できない方やビジュアルスクリプティングのみでゲームを作られている方にはおすすめしません
触る部分のプログラムに関しては非常に簡単なものになっています(各ノードの実行方法ページ参照)

▼基本

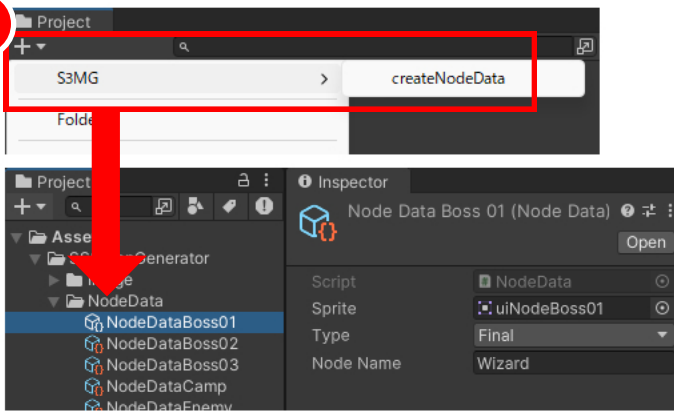


<デフォルトのマップルール>

- Pathは交差しない:設定で変更可
- 同じNodeは極力連続して配置しない:設定で変更可能
ただし設定次第ではルールが破綻する可能性があるので最低1つは連続可能Nodeの登録必須、それでも破綻するならランダムで割り当て
- 分岐があるNodeの場合、行き先は必ず違うタイプのNodeにする:設定で変更可能
ただし設定次第ではルールが破綻する可能性があるので、その場合はランダムで割り当て
- マップルールが当てはまらない設定の場合は、生成不可回避の為一定の試行回数を超えると諦めてランダムでNodeを割り当て

▼クイックスタート

1



Projectの

S3MG > createNodeData

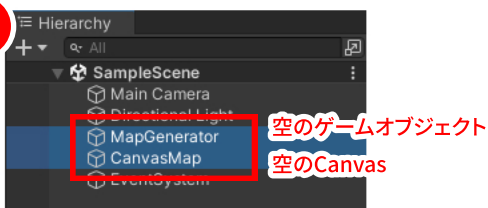
をクリックし、好きな数だけノードデータを作ります

これが各ステージになります

ノードデータの設定に関しては次のページで説明しますが

Node Nameは空白でも大丈夫です

2



ヒエラルキーに適当な「空のゲームオブジェクト」と「空のCanvas」を作ります

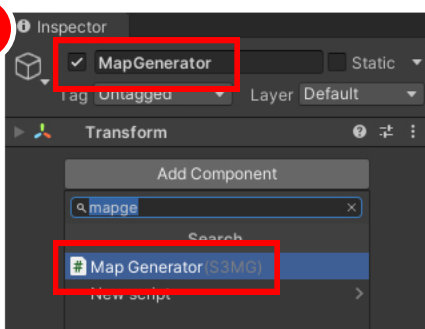
左の図ではそれぞれ

• MapGenerator

• CanvasMap

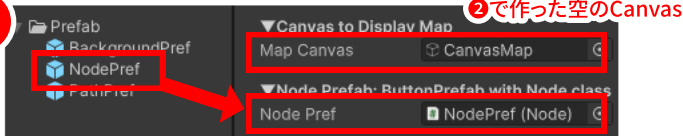
と名前をつけました

3



作った「空のゲームオブジェクト」にMapGenerator.csを追加します

4



インスペクタに必須項目を設定します

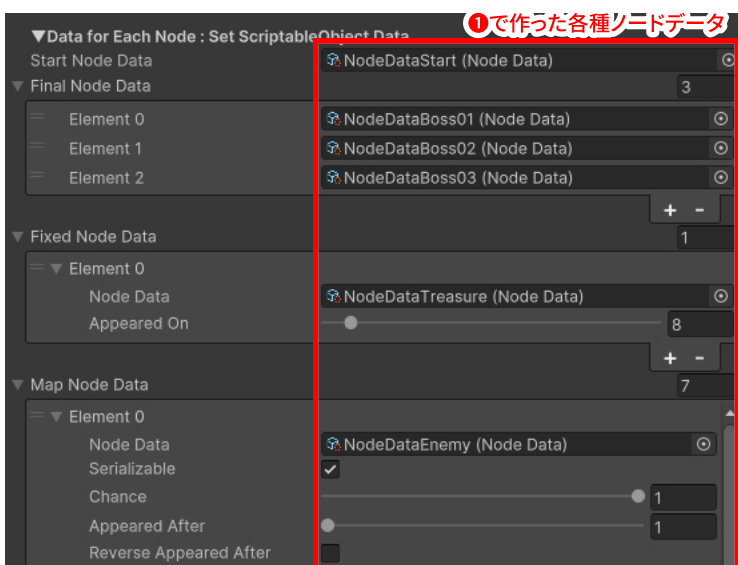
必須項目は以下の3つです

• Map Canvas

• Node Pref

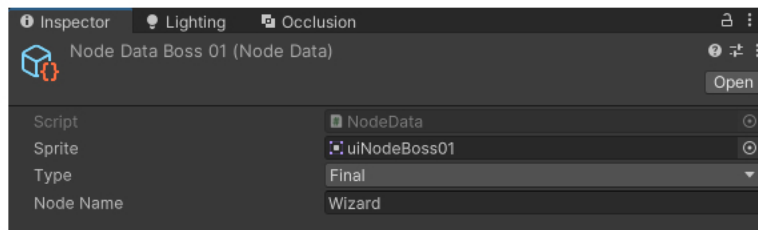
• 各種ノードデータ

他の設定に関しては後述します



以上で、ゲームを開始すればマップが生成されます

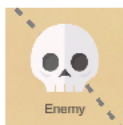
▼createNodeDataに関して



<Sprite>

ノードに表示する画像です

好きな正方形画像を登録してください



<Type>

ノードタイプです

デフォルトでは

Empty, Start, Camp, Shop, Event, Treasure, Trap, Enemy, Middle, Final, Random

の11タイプ登録してあるので好きなタイプを選択してください

上記で足りない場合は、NodeData.csクラスから独自のタイプを追加する事も可能です

もし追加した場合は、Node.csクラスにも処理を追加する必要があります

詳しくは「各ノードの実行方法」の「Extra」の項目で説明します

<Node Name>

ノードの名前です

入力必須ではありませんが、入力しておくマップ上に名前を表示する事が可能になります

また、**同じTypeでも名前をつけておくと別々の処理を行う事が可能になります**

例えば「Type」を「Final」で3つ作成し、「Node Name」を「Warrior」「Wizard」「Ghost」のようにしておけばプログラムで処理を分ける事ができます

▼MapGeneratorの各種設定

Script MapGenerator

▼Skip Node Processing : If true, node processing is skipped, and only the stage advances
Skip Node Processing ☐ ステージの移動だけを確認したい場合はこちらをonにしてください
ノード処理をスキップするかどうか:trueならノードの処理はスキップされ、ステージのみが移動

▼Generate on Execution
Play On Awake ☒ ゲームを実行と同時にMap生成するかどうか

▼Make Start Node
Make Start ☒ スタートノードを作るかどうか

▼Offset Nodes in Placement
Is Offset ☒ ノードをずらして配置するかどうか

▼Show Text on Nodes
Show Text ☒ ノードにテキストを表示するかどうか

▼Allow Path Intersection
Crossable ☐ パスを交差可能にするかどうか

▼Completely Random Placement : Except for fixed nodes, do not apply the default map placement
All Random ☐ 完全ランダム配置:固定ノードを除き、デフォルトのマップ配置ルールを適用しない

▼Whether to randomize the seed : If true, the value of the variable seed is invalid
Randomize Seed ☒ シード値をランダムにするかどうか:trueの場合は変数seedの値無効
Seed 0 シード値:「RandomizeSeed」がfalseの場合、こちらの値が使われる

▼Map Basic Settings
Floor Num 15 フロア数
Route Num 7 ルート数
Normal Node Size 80 通常ノードの大きさ
Start Node Size 160 スタートノードの大きさ
Final Node Size 160 ファイナルノードの大きさ
Floor Distance 80 フロア間の距離
Route Distance 80 ルート間の距離

▼Active Route Count : Limited to RouteNum if exceeded
Active Route Num 4 アクティブルート数:「RouteNum」より多い場合は「RouteNum」に制限

▼Canvas to Display Map
Map Canvas CanvasMap マップを表示するCanvas ★登録必須

▼Node Prefab: ButtonPrefab with Node class
Node Pref NodePref (Node) ノード用Prefab:Nodeクラスを持つButtonPrefab ★登録必須

▼Path Settings : Prefab is not required, Custom Prefab with settings like shaders is also accepted
Path Image Pref PathPref (Image) パスを描画するためのImageプレファブ:なくても可、Shader等を設定した独自のPrefabでも可
Path Color 通常のパスの色と、通過後のパスの色
Passed Path Color
Path Width 4 パスの太さ
Padding Between Nodes 0 ノードとのパディング

▼Background Prefab : Prefab is not required, Image Prefab with 9-slice scaling
Background Pref BackgroundPref 背景Prefab:9スライスが設定されたImageのPrefab:なくても可

▼Padding Between Background and Map
Background Padding 80 背景とマップの余白

▼Mouse Drag Sensitivity and Gamepad Movement Sensitivity
Mouse Sensitive 1 マウスのドラッグ感度
Gamepad Sensitive 2 ゲームパッドの移動感度

▼Data for Each Node : Set ScriptableObject Data
Start Node Data NodeDataStart (Node Data) スタートノードデータ:1つだけ ★「MakeStart」が「true」の場合は登録必須
Final Node Data 3
Element 0 NodeDataBoss01 (Node Data) ファイナルノードデータ:★最低1つは登録必須
Element 1 NodeDataBoss02 (Node Data) 複数登録すればその内の1つがランダムで選択される
Element 2 NodeDataBoss03 (Node Data)

▼Fixed Node Data 1
Element 0
Node Data NodeDataTreasure (Node Data) 固定ノードデータ:複数登録可、なくても可
Appeared On 8 「Appeared on」に設定したフロアに確定で作られるが、
複数登録されていて同じフロアが設定されていれば後の方を優先

▼Map Node Data 7
Element 0
Node Data NodeDataEnemy (Node Data) マップノードデータ:複数登録推奨 ★最低1つは登録必須
Serializable ☒ 連続で配置できるか
Chance 1 出現確率
Appeared After 1 どのフロア以降から登場するか
Reverse Appeared After ☐ 「appearedAfter」を反転するか:反転すると「appearedAfter」に設定したフロア以降には登場しない
Element 1
Node Data NodeDataCamp (Node Data)

▼各ノードの実行方法

```
/*-----  
Processing each node  
-----*/  
void handleStart(){  
    Debug.Log("Start the game");  
}  
  
void handleCamp(){  
    Debug.Log("Open a camp");  
}  
  
void handleShop(){  
    Debug.Log("Open a shop");  
}  
  
void handleEvent(){  
    Debug.Log("Execute event");  
}  
  
void handleTreasure(){  
    Debug.Log("Obtain treasure");  
}  
  
void handleTrap(){  
    Debug.Log("Execute trap");  
}  
  
void handleEnemy(){  
    Debug.Log("Fight the enemy");  
}  
  
void handleMiddle(){  
    Debug.Log("Midpoint");  
}  
  
void handleFinal(){  
    switch(nodeText.text){  
        case "Boss01" :  
            Debug.Log("Fight Boss 01");  
            break;  
        case "Boss02" :  
            Debug.Log("Fight Boss 02");  
            break;  
        default :  
            Debug.Log("Fight Boss 03");  
            break;  
    }  
}  
  
void handleRandom(){  
    Debug.Log("Execute random process");  
}
```

Node.csクラスを開くと下の方に左図の部分があります

ここに対応するノードの処理を書きます

各処理にDebug.Logを記述しているのでそちらを好きな処理に書き換えてください

handleFinal()部分を見ていただくとわかりますが、ファイナルノードの中のSwitch文で処理を分けています

「createNodeDataに関して」のページでも解説しましたが、同じTypeでもNodeNameによって処理を分ける事ができます

<超重要>

各ノードの処理で次のノードに移動したりマップを表示・非表示にしたいタイミングが出てくると思います

その際はMapGenerator.csクラスの以下のメソッドを実行します

シングルトンで実装されていますので、instanceを介して呼び出してください

- toNextNode() 次のノードに移動するメソッド
- activeMap() マップを表示するメソッド
- inactiveMap() マップを非表示にするメソッド
- init() 「playOnAwake」が「false」の時、手動でマップを生成するメソッド
- reGenerate() マップを再構築するメソッド

例：移動先のシーンでマップを非表示にする

using S3MG;

public class SampleStartSceneManager : MonoBehaviour{

```
    void Start(){  
        MapGenerator.instance.inactiveMap();  
    }  
}
```

<Extra>

「createNodeDataに関して」のページで少し話しましたが、独自のタイプを追加する事も可能です

追加したい場合はまずNodeData.csクラスのenumに独自のタイプを追加します

その後、追加したenumの値をNode.csクラスの「entryStart()」内のSwitch文にも追加してください

合わせてhandle***()も記述すればokです

▼各Prefabについて



Prefabフォルダ内に入っているPrefabについて説明します

<BackgroundPref>

背景用のPrefabで、Imageコンポーネントだけが設定されています

「MapGenerator」の「Background Pref」に登録するPrefabですが、**使用は任意です**

登録しない場合、背景はSkyboxやカメラに設定されたカラーがそのまま表示されます

マップの広さに応じて広がるように9-sliceが設定されていますので、

もし独自の背景にしたい場合は9-sliceの画像でImageコンポーネントの「Source Image」を差し替えてください

<NodePref>

Node用のPrefabで、ButtonコンポーネントとAudioSourceコンポーネントが設定されていてNode.csクラスを持っています

「MapGenerator」の「Node Pref」に登録するPrefabで、**必須です**

Buttonコンポーネントの「Highlighted Color」や「Selected Color」は好きなカラーに変更可能です

AudioSourceコンポーネントの「AudioClip」はノードをクリックした時の音なので自由に変更可能です

Node.csクラスの「Visit Image」はノードをクリックした時にノードにアニメーションでつくマークで、自由に変更可能です

Node.csクラスの「Lerp Duration」は点滅速度、「Lerp Color」は点滅色で、自由に変更可能です

<PathPref>

Path用のPrefabで、Imageコンポーネントだけが設定されています

「MapGenerator」の「Path Image Pref」に登録するPrefabですが、**使用は任意です**

こちらに登録すると点線が使えますが、使用しない場合は単純な線になります

点線以外の独自の画像を設定する事もできますが、例えば点線を動かすような独自のシェーダーを設定した画像なども設定できます

独自シェーダーを作れる方はPathをアニメーションさせると面白い表現が可能です

ただし、その場合は「MapGenerator」の「Path Color」や「Path Width」等は無効になります

▼リリースノート

詳しい内容はアセットストアのリリースログをご覧ください

▼免責

このアセットを使用して生じた不具合については、一切の責任を負いかねますのであらかじめご了承ください

▼クレジット

This assets was developed by loloop (C) All rights reserved.